Name: Cohort/Batch: Date: Score:

PL/I PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of PL/I? Threat Model

Q2 How does PL/I handle type checking: static or dynamic? Architecture

Q3 How does PL/I manage memory? Configuration

Q4 What is PL/I's concurrency model? Hardening

Q5 How are packages and dependencies managed in PL/I? SSO / IdP

Q6 How does PL/I implement object-oriented or functional programming? Observability

Q7 How does PL/I handle errors? Lifecycle

Q8 What testing tools are commonly used in PL/I? Compliance

Q9 What makes PL/I well-suited for its target domain? Testing

Q10 What are common performance pitfalls in PL/I? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 **PL/I was designed with specific priorities around** Mitigation

PL/I was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A2 **PL/I uses its type system to catch** Primitives

PL/I uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A3 **PL/I uses one of three approaches: manual** Hardening

PL/I uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A4 **PL/I supports concurrency through threads, coroutines,** Gotchas

PL/I supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A5 **PL/I uses a package manager and a** Federation

PL/I uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A6 **PL/I supports classes and inheritance for OOP,** Observability

PL/I supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern PL/I programs blend both paradigms depending on the problem.

A7 **PL/I surfaces errors via exceptions, Result/Either types,** Automation

PL/I surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A8 **PL/I has a standard or widely adopted** Governance

PL/I has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A9 **PL/I excels in its domain due to** Assessment

PL/I excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A10 **Typical performance issues in PL/I include excessive** Optimization

Typical performance issues in PL/I include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.